

Penetration Test Report — Drone Management System

Datum: 2026-04-15
Tester: Matthias Teuschl
Kurs: Secure Systems Engineering (SSYE) — FH Technikum Wien
Target: http://127.0.0.1:3000
Git Commit: 5b7b2cc
Node.js: v24.12.0 | **Express:** 4.21.2 | **MariaDB:** 11 (Docker)

Executive Summary

Das Drone Management System zeigt insgesamt eine solide Sicherheitsarchitektur. Grundlegende Schutzmechanismen wie CSRF-Schutz, Session-Regeneration nach Login, Passwort-Hashing mit bcrypt (cost 12) + HMAC-SHA256-Pepper sowie parameterisierte Datenbankabfragen sind korrekt implementiert.

Es wurden **3 bestätigte Findings** und **1 weiteres aus Code-Review** identifiziert, davon kein kritisches. Die gravierendsten Punkte sind das vollständige Fehlen eines Content-Security-Policy-Headers und die ungewollte Offenlegung interner Stack-Traces bei CSRF-Fehlern.

Findings Übersicht

ID	Titel	Schweregrad	OWASP	Status
F-01	Fehlender Content-Security-Policy Header	Medium	A05	Bestätigt
F-02	Fehlender Permissions-Policy Header	Low	A05	Bestätigt
F-03	Stack-Trace Offenlegung bei CSRF-Fehler	Medium	A05	Bestätigt
F-04	Template-Fehler im CSRF-Error-Handler	Low	A05	Code-Review
F-05	Timing Oracle bei Login (fehlende bcrypt-Dummy)	Low	A07	Code-Review

Detaillierte Findings

F-01: Fehlender Content-Security-Policy Header

Schweregrad: Medium
OWASP: A05 – Security Misconfiguration
Quelle: src/app.js:22

Beschreibung:
Der Content-Security-Policy (CSP) Header ist explizit deaktiviert. CSP ist eine der wichtigsten browserseitigen Abwehrmechanismen gegen Cross-Site-Scripting (XSS). Ohne CSP werden XSS-Payloads, die server-seitige Output-Encoding umgehen, uneingeschränkt im Browser ausgeführt.

Evidence:

```
# Nikto-Scan:
[013587] /: Suggested security header missing: content-security-policy

# Curl-Header-Prüfung:
$ curl -s -D - http://127.0.0.1:3000/login -o /dev/null | grep -i "content-security"
[kein Output – Header fehlt]
```

Code-Stelle:

```
// src/app.js:21-23
app.use(helmet({
  contentSecurityPolicy: false // <-- explizit deaktiviert
}));
```

Risiko:

Ohne CSP hat ein Angreifer bei einem XSS-Angriff freie Hand im Browser: Cookie-Diebstahl, Keylogging, Weiterleitung auf Phishing-Seiten. In Kombination mit einer XSS-Schwachstelle (z.B. in `Drone-display_name`) wäre der Angriff ungemindert.

Empfehlung:

CSP aktivieren und für diese EJS-Anwendung konfigurieren:

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['"self"'],
      scriptSrc: ['"self"'],
      styleSrc: ['"self"'],
      imgSrc: ['"self"', "data:"],
      frameAncestors: ['"none"']
    }
  }
}));
```

F-02: Fehlender Permissions-Policy Header

Schweregrad: Low

OWASP: A05 – Security Misconfiguration

Quelle: `src/app.js` (Helmet-Konfiguration)

Beschreibung:

Der `Permissions-Policy` Header fehlt. Dieser Header erlaubt es, Browser-APIs wie Kamera, Mikrofon oder Geolocation für die Seite zu deaktivieren.

Evidence:

```
# Nikto-Scan:
[013587] /: Suggested security header missing: permissions-policy
```

Risiko: Low — keine direkte Angriffsmöglichkeit, aber Defense-in-Depth-Lücke.

Empfehlung:

```
app.use(helmet({
  permissionsPolicy: {
    features: {
      camera: ['none'],
      microphone: ['none'],
      geolocation: ['none']
    }
  }
}));
```

F-03: Vollständiger Stack-Trace bei CSRF-Validierungsfehler

Schweregrad: Medium

OWASP: A05 – Security Misconfiguration

Quelle: `src/security/csrf.js:60` + `src/views/layout.ejs:16`

Beschreibung:

Wenn die CSRF-Validierung fehlschlägt, versucht der Error-Handler das `layout.ejs` -Template zu rendern. Da zu diesem Zeitpunkt der Middleware-Stack die User-Kontext-Variablen (`isAdmin`, `isSuperAdmin`) noch nicht gesetzt hat, schlägt das Rendering fehl. Das Ergebnis ist ein HTTP 403 mit vollständigem Node.js Stack-Trace im Response-Body, der interne Dateipfade, Variablenamen und Template-Code preisgibt.

Evidence:

HTTP/1.1 403 Forbidden

```
<pre>ReferenceError: .../src/views/layout.ejs:16
isAdmin is not defined
    at eval (eval at compile (.../node_modules/ejs/lib/ejs.js:673:12)...)
    at csrf_protect (.../src/security/csrf.js:60:25)
    ...
</pre>
```

Offengelegte Informationen:

- Vollständiger Dateisystempfad: `/Users/matthiasteuschl/Library/Mobile Documents/com~apple~CloudDocs/FH Technikum/MCS/02_SS26/SSYE/proj2/`
- Interne Modulstruktur (EJS, Express Versionen)
- Template-Code-Zeilen

Ursache:

Middleware-Reihenfolge in `src/app.js` :

```
app.use(createSessionMiddleware());
app.use(attach_csrf_token);
app.use(csrf_protect);           // <-- läuft VOR user-context middleware

app.use((req, res, next) => {
```

```
res.locals.isAdmin = ...; // <-- wird erst HIER gesetzt
res.locals.isSuperAdmin = ...;
});
```

Wenn `csrf_protect` eine 403-Antwort rendert, sind `isAdmin` / `isSuperAdmin` in `res.locals` noch nicht definiert.

Empfehlung:

Option A — User-Kontext-Middleware VOR den CSRF-Middleware setzen:

```
app.use(createSessionMiddleware());
app.use((req, res, next) => { // User-Kontext zuerst
  res.locals.user = ...;
  res.locals.isAdmin = ...;
  res.locals.isSuperAdmin = ...;
  res.locals.csrfToken = '';
  next();
});
app.use(attach_csrf_token);
app.use(csrf_protect);
```

Option B — Im CSRF-Error-Handler alle benötigten Template-Variablen explizit übergeben:

```
res.status(403).render('layout', {
  title: 'Forbidden',
  isAdmin: false,
  isSuperAdmin: false,
  csrfToken: '',
  ...
});
```

Option C (empfohlen für Production) — `NODE_ENV=production` setzen, um Stack-Traces in Express automatisch zu unterdrücken.

F-04: Timing Oracle beim Login (nicht-existenter User überspringt bcrypt)

Schweregrad: Low

OWASP: A07 – Identification and Authentication Failures

Quelle: `src/routes/auth.js:75–77` (Code-Review)

Beschreibung:

Wenn ein Login-Versuch mit einer nicht-existenten E-Mail-Adresse erfolgt, wird der bcrypt-Vergleich übersprungen und die Route antwortet sofort. Bei einem existenten User mit falschem Passwort wird bcrypt (cost 12, ~100–400ms) ausgeführt. In der Theorie ermöglicht dieser Zeitunterschied User-Enumeration via Timing-Angriff.

Test-Ergebnis:

```
Nicht-existenter User (5 Messungen): ø 11ms
Existenter User, falsches Passwort (5 Messungen): ø 11ms
Differenz: 0ms
```

Der Test konnte die theoretische Differenz praktisch nicht bestätigen (wahrscheinlich weil bcrypt bei dieser Testumgebung den Zeitunterschied in der Netzwerklatenz versteckt, oder kein echter User mit Passwort vorhanden war).

Code-Stelle:

```
// src/routes/auth.js (vereinfacht)
const row = await findUserByEmail(email);
if (!row) {
    return res.redirect('/login?err=1'); // sofortiger Return, kein bcrypt
}
const match = await bcrypt.compare(hmac(password), row.password_hash);
```

Empfehlung:

Dummy-bcrypt-Vergleich auch bei nicht-existenten Usern durchführen:

```
if (!row) {
    await bcrypt.compare('dummy',
'$2b$12$invalidhashpadding00000000000000000000000000000000');
    return res.redirect('/login?err=1');
}
```

Getestete und bestandene Punkte

Test	Erwartet	Ergebnis
SQL Injection auf /login (manuell + sqlmap)	Keine Injection möglich	✓ PASS
SQL Injection auf /forgot (manuell + sqlmap)	Keine Injection möglich	✓ PASS
Time-Based SQLi via SLEEP(3) auf /forgot	Kein Delay erkennbar	✓ PASS
CSRF ohne Token (POST /login)	HTTP 403	✓ PASS
CSRF mit leerem Token	HTTP 403	✓ PASS
CSRF mit falschem Token (64-char hex)	HTTP 403	✓ PASS
CSRF Cross-Session (Token von Session A in B)	HTTP 403	✓ PASS
Session Fixation	Session-ID nach Login neu	✓ PASS
Session Invalidierung nach Logout	Alter Cookie → 302 /login	✓ PASS
Rate Limiting auf /login	HTTP 429 ab Versuch 11	✓ PASS
Unauthenticated Access /dashboard	302 → /login	✓ PASS
Unauthenticated Access /admin	302 → /login	✓ PASS
Unauthenticated Access /admin/users	302 → /login	✓ PASS
Unauthenticated Access /admin/drones	302 → /login	✓ PASS

Unauthenticated Access /super-admin	302 → /login	✓ PASS
Unauthenticated Access /my-drones	302 → /login	✓ PASS
Reflected XSS via ?msg=<script>	Kein Script-Tag reflektiert	✓ PASS
Reflected XSS via ?err=	Kein onerror reflektiert	✓ PASS
TRACE HTTP Method	HTTP 404 (nicht erlaubt)	✓ PASS
Sensitive Files (/env, /package.json)	HTTP 404	✓ PASS
node_modules via HTTP	HTTP 404	✓ PASS
X-Powered-By Header	Nicht vorhanden	✓ PASS
X-Frame-Options Header	SAMEORIGIN	✓ PASS
X-Content-Type-Options Header	nosniff	✓ PASS
Strict-Transport-Security Header	max-age=31536000	✓ PASS
Referrer-Policy Header	no-referrer	✓ PASS
Passwort-Hashing (DB-Inspektion)	\$2b\$12\$... (bcrypt cost 12)	✓ PASS
Password Reset Token (DB-Inspektion)	Nur SHA-256 Hash gespeichert	✓ PASS
Session Cookie HttpOnly Flag	Vorhanden	✓ PASS
Session Cookie SameSite=Lax Flag	Vorhanden	✓ PASS

Nicht getestete Punkte (Einschränkungen)

Test	Grund
Stored XSS via display_name	Drone-Erstellung fehlgeschlagen (CSRF-Session-Problem mit curl)
Privilege Escalation admin → super_admin	Kein dedizierter admin-Role User vorhanden (nur super_admin)
IDOR auf /my-drones	Kein nicht-privilegierter User mit Drone-Zugriff angelegt
force_pw_change Enforcement	Benötigt neu erstellten User mit gesetztem Flag
Password Reset Token Wiederverwendung	Kein SMTP konfiguriert, Token-Log nicht verfügbar

Tool-Ergebnisse

Tool	Ergebnis
Nikto	2 fehlende Security-Header (CSP, Permissions-Policy), sonst unauffällig

sqlmap	Keine SQL-Injection gefunden (parameterisierte Queries bestätigt)
curl	CSRF, Session, Auth und Header-Tests durchgeführt

Testumgebung

Komponente	Version
Node.js	v24.12.0
Express	4.21.2
MariaDB	11 (Docker)
Nikto	2.6.0
sqlmap	aktuell (Homebrew)
Testdatum	2026-04-15
Tester-Host	macOS Darwin 25.3.0

Severity Klassifikation

Level	Definition
Critical	Direkter Zugriff auf alle Daten / Auth-Bypass / RCE
High	Erheblicher Schaden für Subset der User (z.B. Stored XSS, Priv. Escalation)
Medium	Schutzmechanismus fehlt, Angriff erfordert Vorbedingungen
Low	Defense-in-Depth Lücke, geringe Ausnutzbarkeit
Informational	Kein direktes Risiko, Best-Practice-Empfehlung